

11. ROS2 Parameter Service Case

Note: The accompanying reference video uses the ROS Humble version, for reference only

1. Parameter Overview

Similar to global variables in C++ programming, parameters can facilitate sharing certain data across multiple programs. Parameters are a global dictionary in the ROS robot system that allows data sharing across multiple nodes.

In the ROS system, parameters exist as a global dictionary. What is a dictionary? Just like a real dictionary, it consists of a name and a value, also called key and value, combined as key-value pairs. Alternatively, we can understand it as being similar to parameters in programming. There is a parameter name, followed by an equals sign, and then the parameter value. When using it, you just access the parameter name.

Parameters have rich features. For example, if a node shares a parameter, other nodes can all access it. If a node modifies a parameter, other nodes can immediately detect the change and get the latest value.

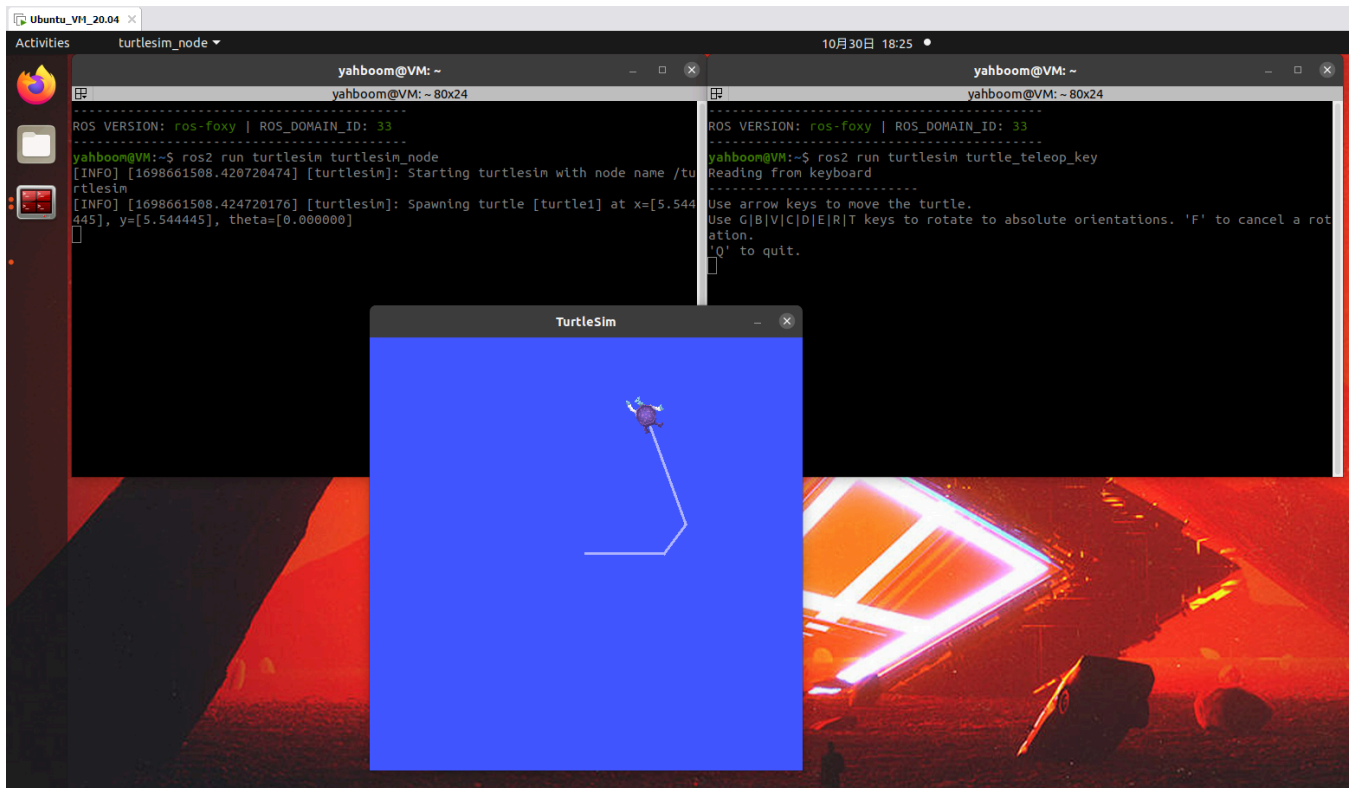
2. Parameters in TurtleSim Example

In the turtlesim example, the simulator also provides several parameters. Let's familiarize ourselves with the meaning of parameters and command-line usage through this example.

Since this requires displaying a GUI, for convenience, the following examples are demonstrated in the virtual machine that comes with this course.

1. Start two terminals in the virtual machine and run the turtlesim simulator and keyboard control node respectively:

```
ros2 run turtlesim turtlesim_node  
ros2 run turtlesim turtle_teleop_key
```



2. Start a terminal in the virtual machine and use the following command to view the parameter list:

```
ros2 param list
```

```

yahboom@VM: ~
yahboom@VM: ~ 80x24
-----
ROS VERSION: ros-foxy | ROS_DOMAIN_ID: 33
-----
yahboom@VM:~$ ros2 param list
/progress_action_server:
  use_sim_time
/teleop_turtle:
  scale_angular
  scale_linear
  use_sim_time
/turtlesim:
  background_b
  background_g
  background_r
  use_sim_time
yahboom@VM:~$

```

3. Parameter Query and Modification

If you want to query or modify a parameter's value, you can follow the param command with get or set subcommands:

```
ros2 param describe turtlesim background_b    # View description of a parameter
ros2 param get turtlesim background_b         # Query a parameter's value
ros2 param set turtlesim background_b 10      # Modify a parameter's value
```

4. Parameter File Save and Load

It's troublesome to query/modify parameters one by one. Let's try parameter files. Parameter files in ROS use YAML format. You can follow the param command with the dump subcommand to save all parameters of a node to a file, or use the load command to load all content from a parameter file at once:

```
ros2 param dump turtlesim >> turtlesim.yaml # Save a node's parameters to a parameter file
ros2 param load turtlesim turtlesim.yaml    # Load all parameters from a file at once
```

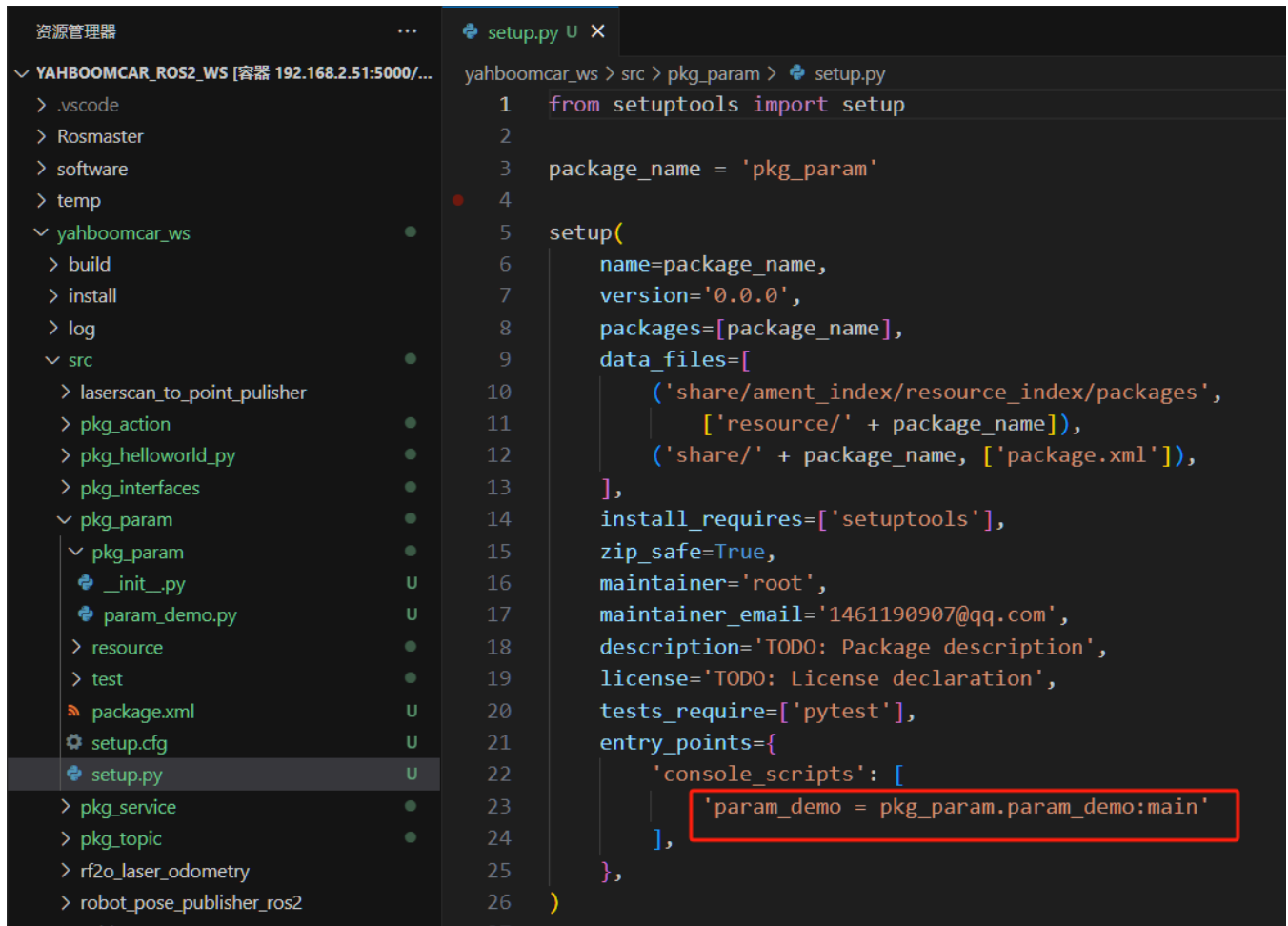
3. Parameter Example

3.1. Create New Package

- Create a new package in the workspace src directory:

```
ros2 pkg create pkg_param --build-type ament_python --dependencies rclpy --node-name param_demo
```

After executing the above command, the pkg_param package will be created, along with a param_demo node, and the related configuration files will already be set up.



3.2. Code Implementation

Next, edit `[param_demo.py]` to implement the functionality. Add the following code:

```
import rclpy # ROS2 Python interface library
from rclpy.node import Node # ROS2 Node class

class ParameterNode(Node):
    def __init__(self, name):
        super().__init__(name) # ROS2 node parent class

    initialization
        self.timer = self.create_timer(2, self.timer_callback) # Create a timer (periodic
        callback in seconds)
        self.declare_parameter('robot_name', 'muto') # Create a parameter and
        set its default value

    def timer_callback(self): # Create timer periodic
        callback function
            robot_name_param =
self.get_parameter('robot_name').get_parameter_value().string_value # Read parameter value
from ROS2 system
            self.get_logger().info('Hello %s!' % robot_name_param) # Output log info, print
            the read parameter value
```

```
def main(args=None):
    rclpy.init(args=args)
    node = ParameterNode("param_declare")
    initialize
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

# ROS2 node main entry point
# ROS2 Python interface initialization
# Create ROS2 node object and
# Loop waiting for ROS2 exit
# Destroy node object
# Shutdown ROS2 Python interface
```

3.3. Build Package

```
colcon build --packages-select pkg_param
```

A terminal window titled 'yahboom@yahboom-virtual-machine: ~/workspace' shows the execution of 'colcon build --packages-select pkg_param'. The output indicates the package was built successfully, with a warning about the 'install-scripts' parameter. The summary shows 1 package finished in 0.86s.

```
yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine: ~/workspace 87x24
yahboom@yahboom-virtual-machine:~/workspace$ colcon build --packages-select pkg_param
Starting >>> pkg_param
/usr/lib/python3/dist-packages/setuptools/dist.py:723: UserWarning: Usage of dash-separated 'install-scripts' will not be supported in future versions. Please use the underscore name 'install_scripts' instead
  warnings.warn(
Finished <<< pkg_param [0.67s]

Summary: 1 package finished [0.86s]
yahboom@yahboom-virtual-machine:~/workspace$
```

3.4. Run Program

- First, refresh environment variables, then run the node:

```
ros2 run pkg_param param_demo
```

A terminal window titled 'yahboom@yahboom-virtual-machine: ~/workspace' shows the execution of 'source install/setup.bash' followed by 'ros2 run pkg_param param_demo'. The output shows two log messages from the 'param_declare' node, both saying 'Hello muto!'.

```
yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine: ~/workspace 87x24
yahboom@yahboom-virtual-machine:~/workspace$ source install/setup.bash
yahboom@yahboom-virtual-machine:~/workspace$ ros2 run pkg_param param_demo
[INFO] [1757055862.991549659] [param_declare]: Hello muto!
[INFO] [1757055864.982421322] [param_declare]: Hello muto!
```

Open another terminal and set robot_name to robot:

```
ros2 param set param_declare robot_name robot
```

In the terminal, you can see the continuously printed log information. The "muto" is the default parameter value we set. The parameter name is "robot_name". After modifying this parameter via command line, you can see the terminal output has changed accordingly.

```
root@unbutu:~/yahboomcar_ros2_ws/yahboomcar_ws# ros2 run pkg_param param_demo
```

```
[INFO] [1698663731.020611236] [param_declare]: Hello muto!  
[INFO] [1698663733.004621321] [param_declare]: Hello muto!  
[INFO] [1698663735.004378214] [param_declare]: Hello muto!  
[INFO] [1698663737.004495248] [param_declare]: Hello muto!  
[INFO] [1698663739.005414393] [param_declare]: Hello muto!  
[INFO] [1698663741.004875369] [param_declare]: Hello muto!  
[INFO] [1698663743.005174937] [param_declare]: Hello muto!  
[INFO] [1698663745.005287521] [param_declare]: Hello muto!  
[INFO] [1698663747.005402185] [param_declare]: Hello muto!  
[INFO] [1698663749.005432493] [param_declare]: Hello robot!  
[INFO] [1698663751.005418223] [param_declare]: Hello robot!  
[INFO] [1698663753.005454514] [param_declare]: Hello robot!  
[INFO] [1698663755.004704535] [param_declare]: Hello robot!  
[INFO] [1698663757.005454932] [param_declare]: Hello robot!  
[INFO] [1698663759.005436436] [param_declare]: Hello robot!  
[INFO] [1698663761.005411059] [param_declare]: Hello robot!  
[INFO] [1698663763.005444211] [param_declare]: Hello robot!  
[INFO] [1698663765.005400912] [param_declare]: Hello robot!  
[INFO] [1698663767.005384622] [param_declare]: Hello robot!
```

7. 192.168.2.99 (jetson)

```
root@unbutu:~/yahboomcar_ros2_ws/yahboomcar_ws# ros2 param set param_declare robot_name robot
```

```
Set parameter successful
```

```
root@unbutu:~/yahboomcar_ros2_ws/yahboomcar_ws#
```